CISC 326 / CISC/CMPE 322 Software Architecture

April 6, 2026

Architectural Enhancement

Group 25

Authors

Emanuel Caravaggio - 22gmx2@queensu.ca

Damon Clulow - 20djc9@queensu.ca

Raghab Dhungana - 21rd56@queensu.ca

Adam Elamin - 20aae5@queensu.ca

Paul Fernando - 20pf3@queensu.ca

Aditya Shetty - 21ass17@queensu.ca

Abstract

The current safety features in the Gemini CLI rely on the identification of dangerous commands and prompting the user for approval before execution. This report proposes an architectural enhancement to the Gemini CLI in the form of the Stateful Environment Simulation (SES) Engine. This enhancement aims to improve the user experience by reducing risk when running commands with unknown effects.

The SES Engine introduces a simulation environment where potentially dangerous commands can be tested and their effects predicted and analyzed. This simulation environment captures the current system state and executes the command against it. The resulting output is then analyzed by the existing safety features and the user to determine if the command should be executed.

Within this report, the SES Engine and its subcomponents are outlined and two implementations are proposed, a virtual overlay and a containerized sandbox. A SAAM analysis examines the two approaches and determines that while the virtual overlay has increased performance and is more portable, the isolated execution environment provided by the containerized sandbox makes it the preferred approach.

The enhancement is shown to increase the safety of the Gemini CLI, and improve the user experience by reducing risk, while increasing the complexity of the architecture. At a high-level, a new component and control flow is added, and at a low-level the responsibilities and interactions of existing components are increased. Multiple use cases of the new feature are examined in detail, and the testing strategy for the enhancement is outlined to ensure the functionality is implemented according to the specifications and the affected components are not negatively impacted.

Table Of Contents

1. Introduction and Overview **4**
2. New Feature Overview - Stateful Environment Simulation **4**
2.1 Brief Overview of SES Engine 4
2.2 Proposed Integration into the Concrete Architecture 5
2.3 Brief Overview SES Engine Subcomponents 5
2.4 Important Note on the SES Engine 5
3. Implementation Approaches **6**
3.1 In-Process Virtual Overlay (Approach 1) 6
3.2 Containerized Sandbox (Approach 2) 7
4. SAAM Analysis **8**
4.1 Major Stakeholders and their Important NFRs: 8
4.2 Best Approach to Achieve Enhancement 9
5. Impact on System **10**
5.1 Impact on Conceptual Architecture 10
5.1.1 High-Level Conceptual Architecture 10
5.1.2 Low-Level Conceptual Architecture 11
5.2 Impact on System Qualities 11
5.2.1 Security and Safety 11
5.2.2 Performance and Latency 11
5.2.3 Usability, Transparency, and Trust 12
6. Potential Risks **12**
6.1 Complexity Risk 12
6.2 Performance Risk 12
6.3 Correctness Risk 12
7. Use Cases 12
7.1 Use Case #1: Validating a Batch File Refactor 12
7.2 Use Case #2: Validating Shell Configuration and Environment Integrity 13
8. Testing **14**
8.1 Unit Testing 14
8.2 Integration Testing 15
8.3 End-to-End Validation 15
9. Conclusion **15**
10. Lessons Learnt **16**
11. References **16**
12. AI Collaboration Report **16**

1. Introduction and Overview

This report presents an architectural enhancement to the Gemini CLI system through the introduction of a Stateful Environment Simulation (SES) Engine. As previously analyzed in prior reports, the Gemini CLI provides a flexible command-line interface for interacting with LLM's and executing tasks within a user's local environment. While the current system has mechanisms for tool execution and security validation, it lacks the ability to predict and validate the real world effects of environment-modifying operations before they are accomplished. The proposed SES engine addresses this shortcoming through the introduction of a simulation layer that enables "dry-run" execution of potentially dangerous commands. Rather than using only static policy checks before allowing execution, the system is enhanced to simulate changes to the file system and environment variables, analyze the resulting state, and validate outcomes against safety constraints. This alters the system from its current reactive security model to a proactive, outcome focused safety framework.

The SES engine is designed to integrate directly into the core layer of the system, iterating with the orchestration engine, security and execution environment, and the tool execution framework. The engine acts selectively using a classification-based model, which focuses on tasks that have the possibility to modify the environment, rather than simple read-only operations. The SES engine follows a pipe-and-filter architectural style, decomposed into subcomponents that are each responsible for system state transformation and validation throughout the simulation pipeline.

This report investigates two implementation approaches for the SES engine; an in-process virtual overlay and a containerized sandbox. These approaches are measured through a SAAM analysis, and through the consideration of key non-functional requirements of the Gemini CLI system. Furthermore, the impact of the enhancement on both the conceptual and concrete architectures is examined, outlining potential risks and analyzing its effect on system qualities. Through this enhancement, the Gemini CLI evolves into a safer and more transparent system capable of completing complex, AI-driven tasks with greater reliability and improved user trust.

2. New Feature Overview - Stateful Environment Simulation

The new feature being proposed is a Stateful Environment Simulation (SES) Engine. While the current system handles tool execution and codebase investigation, it lacks a dedicated sandbox to preview the side effects of complex shell commands or file modifications before they are committed to the user's actual environment. This feature aligns with the Safety and Security goals present in the concrete architecture of the system, with the aim being to mitigate user-enabled, AI-enacted, actions that would negatively impact the environment.

2.1 Brief Overview of SES Engine

The SES Engine is an enhancement design to provide a "dry-run" execution context for the Gemini model. Currently the Security & Execution module performs policy checks, but it cannot predict the functional outcome of a command (e.g., *"Will this regex actually delete the right files?"*).

The proposed SES Engine would allow the Orchestration Engine to:

1. Snapshot a localized portion of the filesystem or environment variables.
2. Simulate the execution of tool calls (shell scripts, file writes) within a virtualized overlay.
3. Validate the resulting state against the user's intent before requesting final approval.

2.2 Proposed Integration into the Concrete Architecture

The SES Engine would be integrated as a core service within the Core Layer, specifically nested inside the Core Package (~/packages/core/src). The SES Engine would be a direct dependency of the Agentic Reasoning subcomponent. Instead of delegating directly to the Tool System, the scheduler would first route "modifying" tasks through SES for validation. The SES Engine would sit between the Policy module and the Tool Execution Framework. It would provide the Safety module with a predicted state to analyze for violation, not apparent in the raw command text. The SES would utilize the config module to determine simulation boundaries, such as which directories are safe to snapshot or how much memory the simulation can consume.

2.3 Brief Overview SES Engine Subcomponents

The SES Engine itself would follow a Pipe-and-Filter style, where the environment state is transformed through the filters. This architecture and components will be discussed in greater detail in the Implementation section of this report. The following table outlines the actual sub-components of the proposed feature

Component	**Responsibility**
Simulation Controller	The gatekeeper; decides between Bypass, for safe/read only tasks, and Simulate for environment-modifying or "dangerous" tasks.
State Snapshotter	Captures the current process.env and virtual file-tree of the target directory using the fileDiscoveryService.
Virtual Overlay	This would be a copy-on-write layer (essentially creating a pointer to original data). It would intercept fs and child_process calls, ensuring no changes reach the real environment.
Outcome Analyzer	Compare the post-simulation state with the pre-simulation state to generate a diff (e.g., "3 files changed, 1 service restarted").
Integrity Validator	Works with the policy-engine.ts to ensure the simulated outcome hasn't breached safety constraints (e.g., accidental deletion of .git or .env files).

Sub-Component Decomposition of SES Engine

2.4 Important Note on the SES Engine

It should be noted that the proposed SES engine would not be active for all interactions, as running a full virtualization layer for every query would introduce unnecessary latency and

resource overhead. Instead, it would be triggered selectively based on a classification-based activation model, similar to the existing Routing Module logic. This classification would be the primary function of the Simulation Controller sub-component of the SES Engine. When the Terminal Interface forwards a request, the Orchestration Engine evaluates the prompt using its Routing Module. If the intent is categorized as “non-modifying” (e.g., asking for a code explanation or file summary), the SES Engine remains idle.
If the intent of a command is classified as “environment-modifying” only then is the SES Engine activated. These “dangerous” categories include:

- *File System Mutations:* Commands involving rm, mv, sed -i, fs.writeFile operations and more.
- *System Configuration:* Altering environment variables, shell profiles (e.g., .bashrc), or network settings.
- *Process Execution:* Running arbitrary shell scripts or binaries that could have larger, cascading, side effects.

Therefore, the SES Engine would essentially be working as a proactive extension of the existing Security and Execution Environment. In the current concrete architecture, the Policy component determines if a tool requires user confirmation. The SES Engine would be the mechanism providing data for that confirmation. Meaning, if a tool call is marked as “Always Allowed” in ~core/src/policy, the SES may be bypassed to maintain performance. For conditions discussed previously, or for tools requiring verifyPolicy(), the SES Engine is mandated to run first [2].

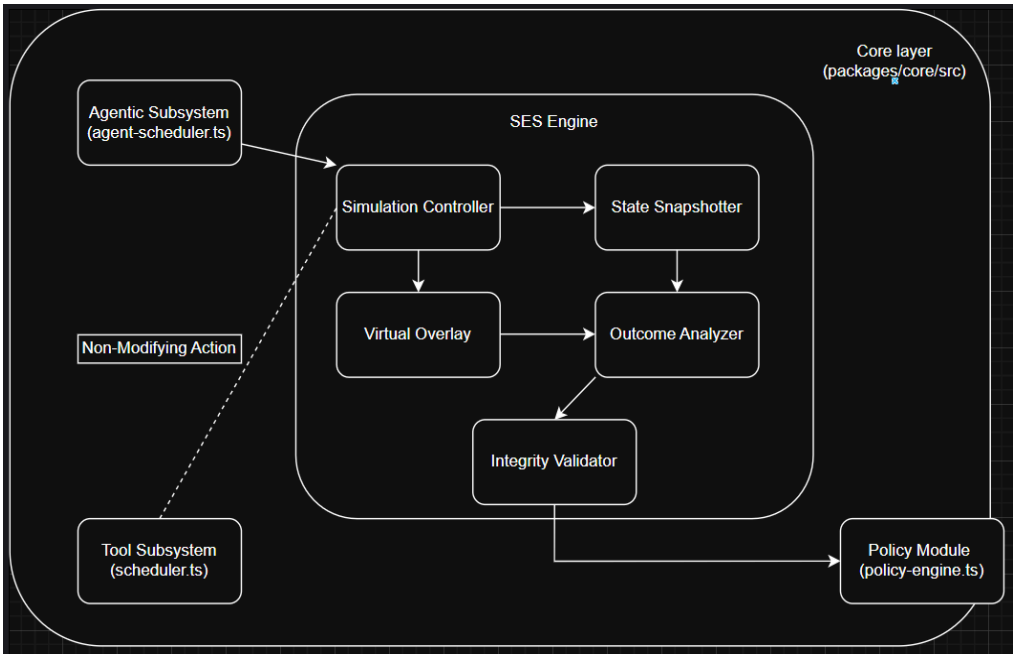
3. Implementation Approaches

We decided to implement a Stateful Environment Simulation (SES) Engine in two approaches. Both approaches are built to predictively execute context for sandboxed shell execution. A key principle shared by both is that the SES Engine must not activate for non-modifying operations, which means that the engine only works to execute commands in the sandboxed shell environment, preserving system performance and responsiveness.

3.1 In-Process Virtual Overlay (Approach 1)

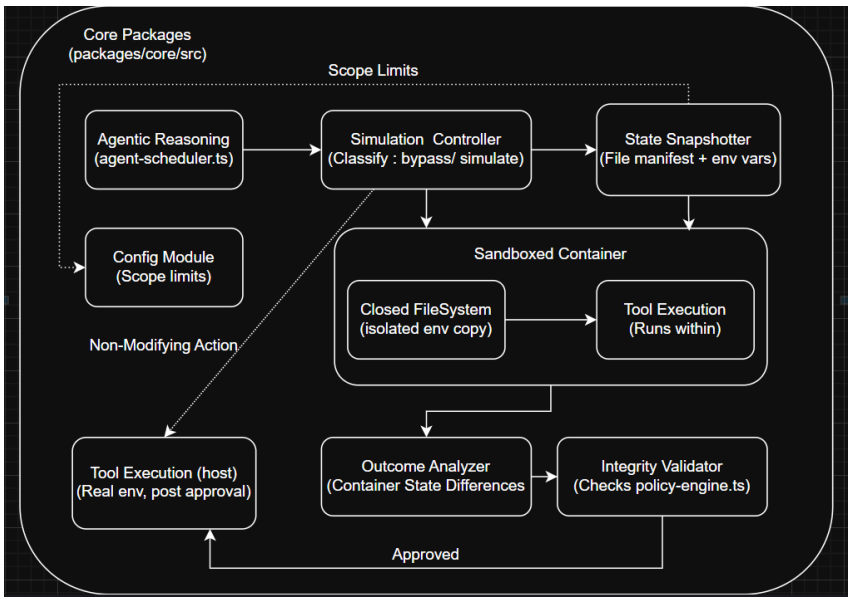
The first approach implements the SES Engine as a layer within packages/core/src alongside that exists and will check against the policy and security file. It works by intercepting Node.js file system and child_process calls at runtime, without needing to compute any external processes. It will work as such a tool call from the Agentic component will be checked against the simulation controller to look at what the tool is actually trying to do. If it passes through the policy and security layer, and is a basic task like reading a file or explaining code, the call will be passed to the Tool Execution Framework as normal. However if it involves modifying files in the environment then the simulation pipeline will begin execution. At that point the State Snapshotter builds an in-memory picture of the target directory using the fileDiscoveryService, which records file paths, permissions and any relevant environment variables, the main distinction is that no files are actually copied [2]. The Virtual Overlay combines the Node.js and child_process modules so that any modification during the simulation are held only in memory and never touch the real file system, this is where the sandboxed element comes into play. Once the summation finishes the Outcome is analyzed and the changes are compared against the original snapshot. This produces a plain-text summary for the user, that displays what changes were made. The Integrity Validator then checks that summary against policy-engine.ts to make sure nothing

dangerous is about to happen, or no core files like .gir or .env files will be tampered with [2]. If everything looks fine, the user sees the changes and decides whether to proceed.
The main upside of this approach is that it is fast and adds very little overhead since it runs entirely within the existing Node.js process. The downside is that native binaries or tools that go around Node.js APIs cannot be fully intercepted, so some side effects might slip through the simulation.



3.2 Containerized Sandbox (Approach 2)

The second approach runs a simulation inside a constraint, like a Docker instance that is created instantaneously and then thrown away when the simulation is over. It acts as a containerized sandbox and is the clear distinction that divides personal files, and the engine's file system. Instead of intercepting calls at the Node.js level, the files and environment variables are copied into the sandbox so that the tool can run inside of it, this includes any shell scripts. The simulation Controller runs tasks through the policy and security files in the same way as Approach 1. When a modifying task is detected the State Snapshotter produces a list of the files and directions that need to be copied. The config module keeps its scope to a certain size, this is so the system does not try to supply a container for an entire codebase. The tool then runs to complete the task inside the container and all of its changes stay contained within it. The Outcome Analyzer compares the container's state against the original snapshot, the Integrity Validator checks the result against the policy-engine.ts , and once approved the real tool call runs in the host environment.[2]
This approach has the advantage of having everything localized within the simulation, so nothing can be changed or altered outside of it. Since the tool runs in a fully isolated environment, every side effect gets captured, including things that Approach 1 might have missed. The trade-offs is that supplying a container takes a longer time than an in-process approach. This is slower in an interactive session. It also requires a container runtime counter to be installed which adds an additional dependency and makes cross-platform support more complex.



4. SAAM Analysis

Based on the two possible implementations of the enhancement, we will consider how these impact major stakeholders and then identify the most important non-functional requirements (NFRs) for this enhancement. The major stakeholders of these two different implementations are the System Developers who implement the core AI orchestration framework, and the End Users who utilize the AI assistant to safely investigate and modify their codebases.

Approach 1: In-Process Virtual Overlay

Approach 2: Containerized Sandbox

4.1 Major Stakeholders and their Important NFRs:

System Developers: The developers responsible for the Core Layer act as the system's gatekeepers. They design and implement the SES Engine, ensure seamless integration with the existing Security & Execution module, and maintain cross-platform compatibility while avoiding excessive technical debt.

Portability & Deployability: The framework must remain easy to distribute and run across various user operating systems without excessive external requirements.

- *Approach 1*: Highly portable. As the Virtual Overlay is built entirely within the Node.js ecosystem, it runs natively wherever the core system runs. Developers do not need to worry about external OS-level dependencies [3].
- *Approach 2*: Lower portability. This approach introduces a hard dependency on an external container runtime. Developers have to account for environments where users cannot install container daemons, complicating cross-platform support and deployment pipelines.

Maintainability:The ease of maintaining the simulation logic as the system scales and as underlying technologies update.

- *Approach 1*: Requires constant maintenance of the interception logic. Because it overrides certain APIs, any future updates or changes to standard Node.js libraries may break the virtual overlay, requiring continuous patching by the developers.
- *Approach 2*: Easier long-term maintenance for environment isolation. The container runtime handles the file system and process isolation natively. So, developers do not need to maintain complex API-interception code, and only need to maintain the logic that provisions the containers [4].

End Users: Users who interact with the AI assistant to write scripts, modify files, and execute codebase commands. They rely on the tool to be both safe and responsive. If the enhancement fails to protect their system or slows down their workflow, they will abandon the tool.

Safety: The primary goal of the SES Engine is to accurately predict outcomes and prevent unintended destructive actions to the host environment.

- *Approach 1*: Provides moderate safety. It effectively traps standard Node.js modifying commands, but it cannot fully intercept native binaries or tools that bypass Node.js APIs to interact directly with the OS. This could result in uncaptured side effects, giving the user a false sense of security.
- *Approach 2*: Provides absolute safety. Because the tool runs inside a fully isolated, constraint-based environment, every side effect is captured. Even direct OS-level binaries are contained, ensuring 100% protection for the user's host machine.

Performance: The AI assistant must remain responsive. The simulation should not add frustrating delays to interactive query sessions.

- *Approach 1*: Extremely fast, because the simulation runs entirely in-memory within the existing Node.js process, and the overhead is minimal. Users will experience near-instantaneous validation of dangerous commands [2].
- *Approach 2*: Introduces noticeable latency as, supplying, spinning up, and copying files into a new container takes significantly longer than an in-process simulation [4]. The interactive sessions feel slower, as latency disrupts the conversational flow of the AI assistant.

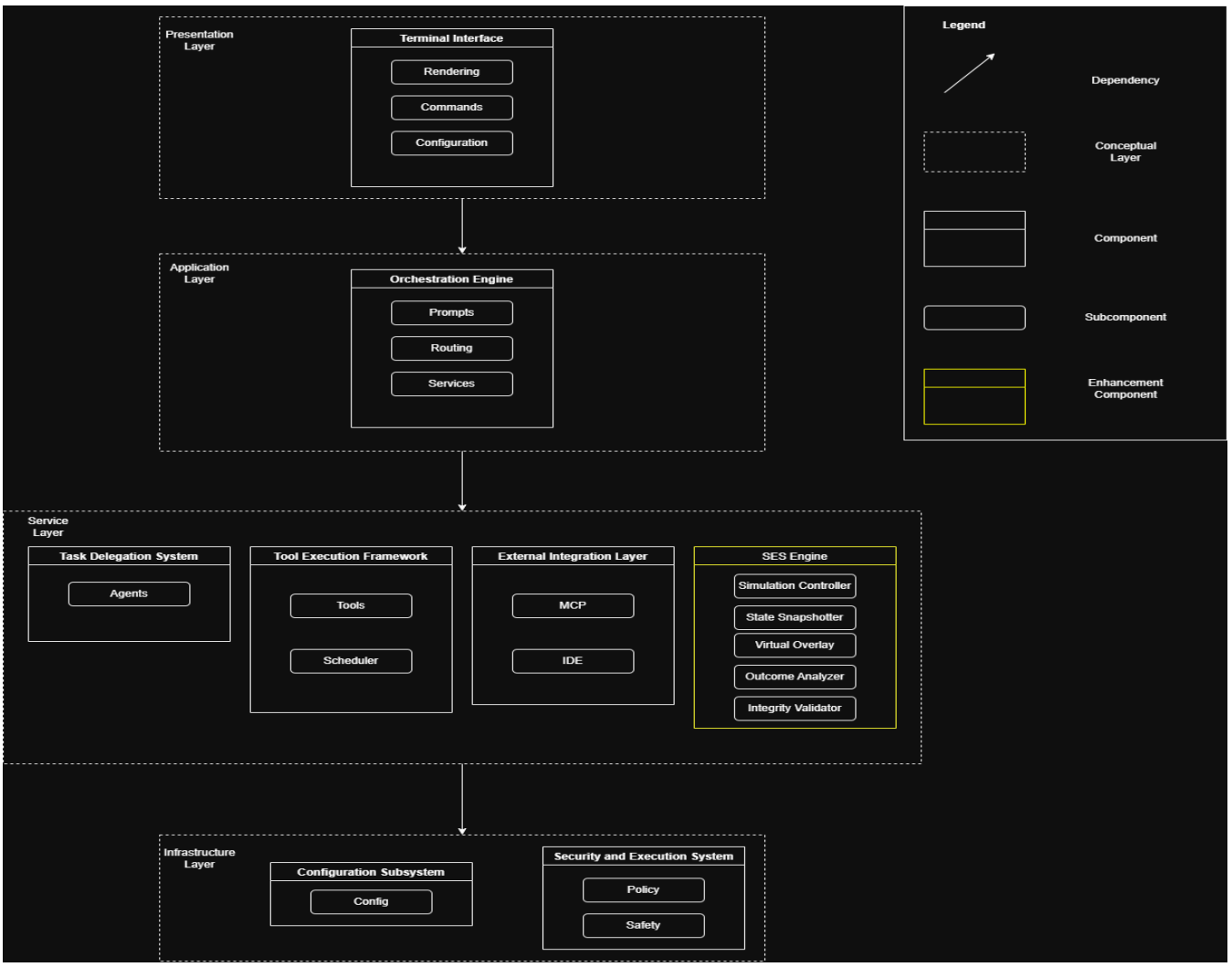
4.2 Best Approach to Achieve Enhancement

Based on the analysis, using a Containerized Sandbox (Approach 2), is the better way to realize the enhancement of adding a Stateful Environment Simulation Engine. While the In-Process Virtual Overlay offers lower latency and easier initial distribution by relying purely on Node.js, it falls short of the core purpose of the enhancement, Safety and Security. The orchestration engine is designed to proactively protect the user from AI-enacted environment modifying tasks, so the simulation layer must be infallible. The inability of approach 1's to intercept native binaries introduces a critical vulnerability where destructive actions could slip through and alter the user's host system. Whereas, Approach 2 aligns perfectly with the concrete architecture's security goals by enforcing strict hardware level isolation [4]. Although the Containerized Sandbox introduces latency due to container spin-up times, the architecture smartly mitigates this through the Simulation Controller, which completely bypasses the SES Engine for safe, read-only tasks, meaning a user only incurs this latency penalty when the AI is attempting a dangerous operation. End users would accept a few seconds of delay in exchange for the absolute guarantee that an AI-generated shell script will not corrupt their files or delete vital source code. Ultimately, Approach 2 provides a more robust, trustworthy, and forward-looking solution for executing untrusted AI commands in a sandboxed state.

5. Impact on System

5.1 Impact on Conceptual Architecture

5.1.1 High-Level Conceptual Architecture



The inclusion of the SES Engine changes the high-level conceptual architecture of the Gemini CLI by adding a new component to the Service Layer, as seen in the conceptual architecture diagram, and by changing the request flow for tasks that would modify the environment. [1] In the current conceptual architecture, a request comes from the user via the Terminal Interface and is passed to the Orchestration Engine, where it is checked by the Security and Execution Framework and then forwarded from the Orchestration Engine to the Tool Execution Framework for execution. With this enhancement, this flow changes as requests that modify the environment would be routed through the SES Engine so that their effects can be simulated before execution. [1]

At the system level, this additional functionality implies additional responsibility. Previously, the system only had to decide if a request was allowed to be executed, but now the system must simulate the request and determine if the likely outcome is dangerous. This will increase the number of steps required for each request as the system must now simulate and validate the outcome of any request that could affect the system.

The architectural enhancement also impacts existing components including the Terminal Interface, Orchestration Engine, Security and Execution Environment, Tool Execution Framework, and Configuration Subsystem as these components must support interaction

with the SES Engine. Overall, this change increases the complexity of the conceptual architecture while creating additional system responsibilities.

5.1.2 Low-Level Conceptual Architecture

The impact on the low-level conceptual architecture includes the introduction of a new simulation engine, and new interactions with existing components. The new SES Engine component has a Pipe-and-Filter architectural style composed of the Simulation Controller, State Snapshotter, Virtual Overlay, Outcome Analyzer, and Integrity Validator. Together they predict the outcome of a request before execution begins.

The entrance to this component is the Simulation Controller. It receives requests from the Orchestration Engine and works with the Security and Execution Environment to determine if the request must be simulated. The State Snapshotter captures a snapshot of the system environment as it is before request execution. The Virtual Overlay provides an isolated execution space so the request can be tested within the environment snapshot without impacting the real environment. The Outcome Analyzer and Integrity Validator then evaluate the predicted outcome to determine if the request might breach any safety constraints.

The SES Engine communicates with additional existing components. The Terminal Interface must render the results of the simulation for the user. The Configuration System provides configurations for the SES Engine, including the resource constraints and the boundaries of the simulation. Finally, the Tool Execution Framework must be extended to support simulated tool execution.

Overall, the low-level impact on the conceptual architecture is an increase in complexity. As a new component is introduced and existing components gain additional responsibilities.

5.2 Impact on System Qualities

5.2.1 Security and Safety

The most significant positive impact of the SES Engine is on the system's security and safety. Previously, the Security and Execution Environment relied solely on static policy checks, to see if a tool was theoretically allowed to run. With the SES Engine, the system can transition to a proactive, based security model. The system can practically guarantee that an AI-enacted command will not result in unintended destructive behaviors, by utilizing the Virtual Overlay and Outcome Analyzer. This would drastically reduce the blast radius of AI hallucinations or poorly formed commands.

5.2.2 Performance and Latency

The most notable negative impact of this enhancement is on system performance. The process of capturing the pre-execution state via the State Snapshotter, simulating the task within the Virtual Overlay, and running the Outcome Analyzer makes this a computationally expensive pipeline, which will increase the time it takes to process requests. However, the system deliberately mitigates this degradation through the Simulation Controller, and preserves high performance and low latency for the majority of interactions.

5.2.3 Usability, Transparency, and Trust

The effect on usability can be viewed in two ways. On one hand, the SES Engine introduces an increase in user friction, and users are now forced to review the simulated outcome via the Terminal Interface and explicitly approve the action before it is committed to the real environment. On the other hand, this friction vastly improves system transparency and user trust. The user is no longer executing AI commands blindly, as the Outcome Analyzer

provides clear, human-readable instructions. This transforms the AI from an unpredictable black-box agent into a safe, predictable assistant.

6. Potential Risks

6.1 Complexity Risk

The addition of the SES engine adds complexity to the conceptual architecture at multiple levels. At a high-level, a component is added and additional control flow paths are introduced depending on the request made. At a low-level, the SES Engine must be implemented, and extensions must be made to existing components to communicate with the SES Engine. These added relationships will make the system harder to reason about. This risk can be mitigated by ensuring a modular implementation of the SES Engine and using well defined interfaces to facilitate communication between the SES Engine and other components.

6.2 Performance Risk

The addition of the SES Engine will require additional processing steps for the completion of requests. The environment simulation, output analysis, and approval process will increase the latency from request to execution, reducing the user experience.
This risk can be mitigated by having the process be highly configurable, the user should be able to choose what type of requests would require simulation, and the resources (compute and memory) that can be used for the simulation.

6.3 Correctness Risk

The addition of the SES Engine introduces a correctness risk, as the user relies on the simulated result to match the real execution. If the output does not align with the actual results, the user might approve actions with dangerous and unexpected side effects. These incorrect results could be caused by many factors including the simulation not having enough resources, or changes in the environment that occurred after the snapshot.
The mitigation strategy for this risk is to ensure the simulation is as thorough as possible and inform the user of the simulation's limitations. This includes flagging potentially dangerous operations, even when the simulation shows no dangerous side effects.

7. Use Cases

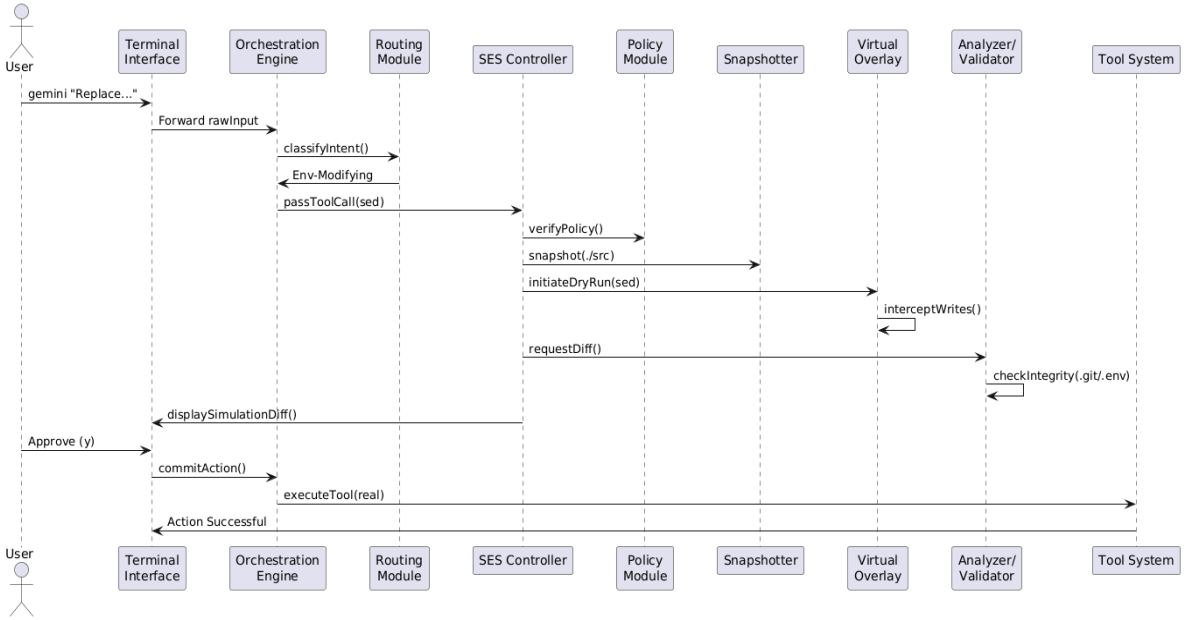
7.1 Use Case #1: Validating a Batch File Refactor

A developer uses the Gemini CLI to perform a complex sed operation across multiple source files. This involved file system mutation, therefore the system must provide a dry-run preview (SES) to ensure the regex does not corrupt the codebase by accident. The following sequence of actions ensues:

1. **Input:** The user enters: gemini "Replace all instances of 'LegacyAuth' with 'ModernAuth' in the src folder."
2. **System Classification:** The Orchestration Engine and its Routing Module identify this as an "environment-modifying" intent.
3. **Simulation Trigger:** The Simulation Controller receives the request and, seeing it is a "dangerous" file mutation, mandates the simulation instead of a bypass.
4. **Simulation Execution**: The SES Engine snapshots the src directory , runs the sed command in the Virtual Overlay, and analyzes the resulting file changes.

5. **Safety Verification**: The Integrity Validator ensures no critical files (like .git or .env) were touched.
6. **User Review**: The user receives a visual diff (e.g., "5 files would be modified") and a safety confirmation.
7. **Final Commitment**: The user approves the changes, and the Orchestration Engine signals the real Tool System to apply them to the actual environment

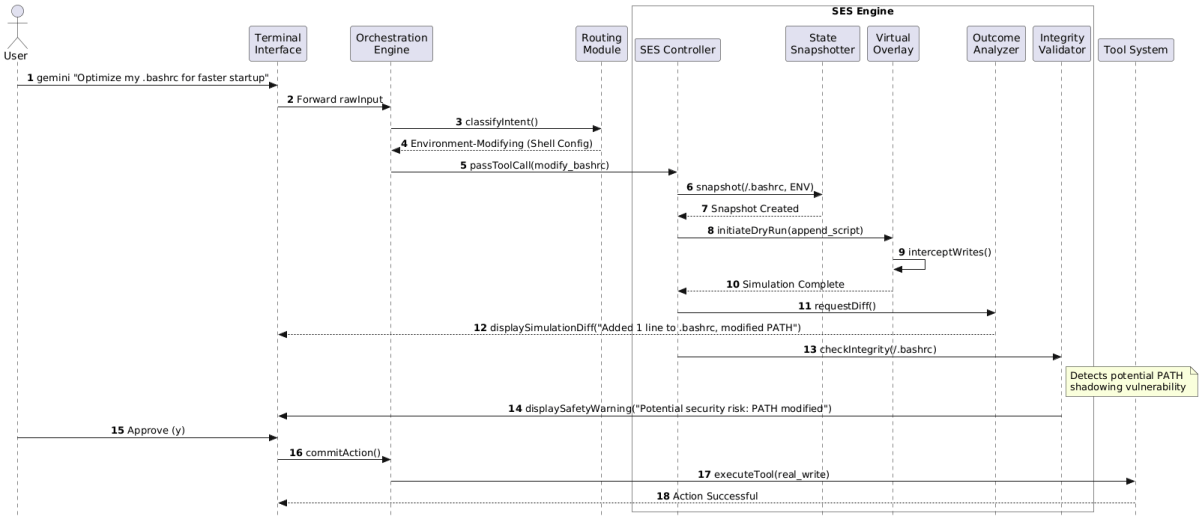
The following sequence diagram depicts the interactions between components in the system and proposed enhancement:



7.2 Use Case #2: Validating Shell Configuration and Environment Integrity

Use Case Description

1. **Input**: The user requests a performance optimization for their shell environment.
2. **System Classification**: The Orchestration Engine identifies the intent as "System Configuration" (a high-risk category).
3. **Simulation Trigger**: The Simulation Controller mandates a dry-run because the target is a protected system file (.bashrc).
4. **Simulation Execution**: The State Snapshotter captures the current shell variables and the .bashrc file. The Virtual Overlay executes the proposed changes in isolation.
5. **Safety & Integrity**: The Outcome Analyzer generates a diff. The Integrity Validator notices that the change modifies the PATH variable in a way that might prioritize a local directory over system binaries (a common security risk), triggering a specific warning.
6. **User Review**: The user sees exactly what lines will be added and receives a warning about the PATH change.
7. **Final Commitment**: Upon user approval, the real Tool System applies the changes to the host environment.



8. Testing

Testing involves seeing how well each individual section of the engine works on its own and how they behave as part of a larger system. Since the SES Engine works between user-approved decision making and a real change in their environment, I will discuss 3 types of testing, unit testing, integration testing and end-to-end validation.

8.1 Unit Testing

Unit tests will check each SES sub-component independently using replicated dependencies to mock the real ones, so external systems do not interfere with the results. The Simulation Controller needs to correctly identify what types of operation the user needs, the test will then run it against the tool calls to see which one is needed. Once the tool commands have been verified and it routes to each command correctly the result will be returned. Edge cases including sed commands and shell-language errors (missing flags, improper piping) will also have tests to confirm that bypassed calls add zero overhead on their way to the Tool Execution Framework. The State Snapshotter will be tested to make sure in-memory structures it builds accurately reflect the real directory at the time of the snapshot and that it respects the configurations in the config module. The Virtual Overlay, will be tested to confirm that nothing escapes to the real life file system during a simulation. After any simulation, irrespective of the user's decision, the file system should remain completely unchanged. The Outcome Analyzer will be tested by feeding it before-and -after state pairs and checking that the differences it produces are accurate. Since these tests are arguably the most important, testing it on file creation, deletion and modification individually as well as operations that combine the three are vital to the engine's fruition. Since the difference is what the user actually sees, the formatting should also be tested. The Integrity Validator will be tested against differences that are known to violate safety constraints which should pass without issue. The goal is to confirm it catches things like accidental deletion of protected paths while not producing false alarms on harmless operations

8.2 Integration Testing

Integration testing checks to see how the SES engine interacts with the rest of the system, across the boundaries where components transfer information to one another. The transfer between the Agentic Reasoning subsystem and the Simulation Controller, is the most important to test. These tests will provide important insight on how tool calls dispatched by

the agent loop are intercepted before reaching the Tool Execution Framework. The results of a simulation should be correctly communicated back so the agent either proceeds or stops. This needs to work for both tool calls and even longer multi-step chains. The SES Engine and the Security and Execution Environment also needs integration testing. The verifyPolicy() function should always be used to check against the SES engine before a final decision is made. These tests will confirm that a violation flagged by the Integrity Validator always blocks the real tool call. Finally, a test for the lifecycle of our container is also needed , this would confirm that the containers get cleaned correctly and that nothing is left behind after a simulation ends, as well that a container failure does not crash or hang the user's session.

8.3 End-to-End Validation

Finally End-to-End Validation tests run the full simulation from start to end. Starting for a user prompt to the final outcome, a core test scenario involves tests that chain multiple shell commands. Tests will require the SES to create, delete and manipulate multiple files over the course of the simulation. The test checks that the SES Engine activates, produces the right differences, waits for user approval, and only then performs the various shell commands on these fields. A paired test confirms that cancelling leaves the file system completely untouched. A simulation that involves a shell script that modifies environment variables, and writes to a log file will have tests that verify that both the environment and file changes show up in the differences and are properly evaluated before running anything. Finally, I will be discussing performance tests, for Approach 1, the latency of the simulation will be measured across different directory sizes to make sure it stays within a reasonable range. For Approach 2, start-up time will be recorded against the latency in the config module. Both approaches will be tested to confirm that non-modifying prompts remain unaffected, and that the SES engine is invisible when not needed.

9. Conclusion

In conclusion, the proposed SES Engine is a meaningful architectural enhancement that addresses a key limitation in Gemini CLI’s ability to safely execute environment-modifying operations. By introducing a simulation layer, the system shifts from static policy checks to an outcome-based safety model, allowing real evaluation of commands to significantly reduce the risk of unintended destructive actions. While the in-process virtual overlay offers performance and portability, it cannot fully guarantee side-effect isolation; conversely, the containerized sandbox provides complete isolation that better aligns with the goal of safe execution. Though the containerized approach adds latency and complexity, these costs can be mitigated through selective activation to maintain satisfactory performance. Ultimately, the SES Engine increases system complexity but achieves enhanced transparency and trust. By presenting clear previews of command outcomes, it empowers users to make informed decisions rather than executing commands without direction. Overall, the SES engine strengthens the Gemini CLI by improving safety, usability, and robustness within the existing architectural design.

10. Lessons Learnt

Designing the SES Engine provided insights into enhancing a production-level AI system while maintaining architectural integrity. A primary takeaway was the distinction between text-based security and outcome-based safety; while the Security and Execution module can

check a command, it cannot predict its actions. Shifting from the static policy-engine.ts to a predictive Virtual Overlay supports the idea that true safety requires a stateful understanding of the filesystem, not just prompt analysis.

We learned that full virtualization for every interaction introduces unacceptable latency, necessitating our classification-based activation model. We also analyzed implementation trade-offs: an In-Process Virtual Overlay offers speed but risks "leaking" side effects, whereas a Containerized Sandbox provides total isolation at the cost of start-up time and runtime dependencies. Ultimately, we found there is no "perfect" architectural choice, only trade-offs between performance and isolation.

Finally, we ensured the SES Engine respected our A2 one-way dependency model. By nesting it in the core package, the presentation layer's independence from the simulation logic was preserved.

11. References

[1] "Global Feedback about A1 - CISC 326 / CISC/CMPE 322 Software Architecture W26." Accessed: Apr. 02, 2026. [Online]. Available: https://onq.queensu.ca/d2l/le/content/1132545/viewContent/6711621/View

[2] *google-gemini/gemini-cli*. (Mar. 26, 2026). TypeScript. google-gemini. Accessed: Apr. 06, 2026. [Online]. Available: https://github.com/google-gemini/gemini-cli

[3] "Index | Node.js v25.9.0 Documentation." Accessed: Mar. 29, 2026. [Online]. Available: https://nodejs.org/docs/latest/api/

[4] "Home," Docker Documentation. Accessed: Apr. 06, 2026. [Online]. Available: https://docs.docker.com/

12. AI Collaboration Report

For our proposed architectural enhancement of Gemini CLI, our team continued to use OpenAI's LLM ChatGPT - 5.2, February 2026 version, as well as Google's Gemini 3, November 2025 version, just as we did to help understand the conceptual and concrete architecture. We chose to use these tools due to their strong capabilities in reasoning about system design, explaining architectural concepts, and assisting with structured analysis tasks. Unlike previous deliverables which focused more heavily on understanding existing system structure, this deliverable required us to design and evaluate an architectural enhancement, which was done through the form of the SES engine. The design of this system required support in refining architectural ideas, comparing implementation approaches, and validating reasoning related to non-functional requirements. We continued our use of these tools over other LLM's because they are widely accessible, reliable ,and when given proper direction, are capable of supporting both an understanding of the high-level and concrete workings of a systems architecture.

Our AI teammate was assigned specific support tasks that aligned with the focus of this report. These tasks were appropriate and specifically selected to ensure that the LLM's contributed to clarification and validation during our design process, rather than generating original design decisions. The set of tasks include:

1. Rephrasing and clarifying descriptions of the SES Engine where we thought necessary.

2. Articulate comparisons between the in-process virtual overlay and the containerized sandbox implementations. This included helping to organize differences in performance, safety and maintainability.
3. Interpret and refine analysis of possible stakeholder concern, through the implementation of non functional requirements, such as safety and performance.

Our prompting strategy we followed focused on providing detailed context and constraining the models responses, while allowing all members of the group to interact with the LLMs collaboratively. We included writing from our own report and design descriptions to ensure that the LLM's output remained aligned with our design. Along with this we instructed the LLMs to avoid introducing external information or assumptions and to base responses only on provided information. An example of a prompt we used was:

- We are designing an architectural enhancement for the Gemini CLI system involving a Stateful Environment Simulation (SES) Engine. Based only on the description provided above, compare the In-Process Virtual Overlay and Containerized Sandbox approaches in terms of safety, and explain which approach better satisfies the system's requirements.

To refine prompting, we added context such as specifying our role as undergraduate students. Additionally, if we received output that we believed did not align with our design, we iterated on the prompt by narrowing the LLM's scope or requesting additional explanations.

To ensure the reliability of outputs, we followed a controlled process. Firstly, all of our prompts were based on information from our own report, course materials and prior deliverables, which made sure that the LLM's responses were contextually relevant and accurate. Secondly, the outputs we received were reviewed by group members before being implemented. If the LLM's output did not align with our design decisions then we would not accept it. Lastly, all outputs were cross referenced with course content, such as the SAAM analysis, architectural styles and non-functional requirements. This ensured that our reasoning remained consistent with the learned concepts in the course.
We believe that our AI teammate contributed approximately 10% of our final deliverable. Our teammates' role was focused on refining explanations and aiding in comparison and analysis, which was not a substantial portion of the deliverable, as the rest members of the team worked on designing the SES engine, developing implementation approaches, completing a SAAM analysis and writing the report. Due to the LLM's role as a support role its contribution was not larger than any individual group member.

The use of our AI teammate had an overall positive impact on our workflow, with some setbacks. One of the main benefits was the ability to quickly refine complex explanations, which was quite helpful when describing architectural concepts and comparing implementation approaches. On the other hand, crafting effective prompts, validating outputs, and refining prompts required time and careful attention, which in certain cases reduced the overall efficiency. Our AI teammate also helped by influencing discussions by providing an additional perspective. While its responses were never treated authoritatively, they usually helped guide further understanding and highlight aspects of our proposal that required more consideration. Through this experience, our team has learned that collaborating effectively with AI can be done by assigning it clear tasks, carefully crafting prompts and through validation of outputs. This approach allowed us to use AI effectively without letting it take control of our design and analysis that we have presented in the report.